

Assignment 2

COM 837, Machine Learning for Wireless Communication Systems

CH Lithin Sai Kumar
IMT2023598

11 September 2026

Code: <https://github.com/ChLithin/com837-assignment-2>

`generate_dataset.py` is unmodified. Everything else is in the two notebooks, `Q1_LOS_NLOS_SVM.ipynb` and `Q2_QAM_Kmeans.ipynb`. All splits use `random_state=42` and all KMeans fits use `n_init=10`, `random_state=42`, so the numbers here reproduce exactly.

1 Q1: SVM for LOS/NLOS identification

Part (a)

From the six tap amplitudes $|h_l| = \sqrt{h_{\text{real},l}^2 + h_{\text{imag},l}^2}$ and the six delays τ_l :

Feature	Definition	Paper Eq.
Kurtosis	$\mathbb{E}[h - \mu]^4 / \sigma^4$	(2)
Skewness	$\mathbb{E}[h - \mu]^3 / \sigma^3$	(5)
Rising time	$\tau(\arg \max_l h_l) - \min_l \tau_l$	(6)
RMS delay spread	$\sqrt{\frac{\sum_l (\tau_l - \tau_m)^2 h_l ^2}{\sum_l h_l ^2}}, \quad \tau_m = \frac{\sum_l \tau_l h_l ^2}{\sum_l h_l ^2}$	(7), (8)
Rician K (dB)	$10 \log_{10} \left(\frac{ h _{\max}^2}{\sum_l h_l ^2 - h _{\max}^2} \right)$	Sec. III-A

Note on the K -factor.

Eq. (9) estimates the diffuse power as $2\sigma_{|h|}^2$, which assumes many delay samples dominated by diffuse energy. With $L = 6$ the dominant LOS tap is one of the six samples and inflates $\sigma_{|h|}^2$ itself, so Eq. (9) returns a lower value for LOS than for NLOS (7.03 dB vs 8.19 dB at 10 dB), which is backwards. I used the power-ratio form above, which is the definition the paper gives in words and the one the sanity check in `generate_dataset.py` uses. It gives 9.61 dB (LOS) and -0.64 dB (NLOS) at 30 dB against generator targets of 9 dB and 0 dB. Both are implemented in the notebook.

Feature	0 dB		30 dB		Separation	
	LOS	NLOS	LOS	NLOS	0 dB	30 dB
Kurtosis	2.548	2.175	3.870	2.164	0.52	3.32
Skewness	0.690	0.301	1.612	0.349	0.64	2.88
Rising time (ns)	3.562	47.640	0.000	28.155	0.90	1.09
RMS delay spread (ns)	20.474	65.372	6.981	45.189	1.60	2.95
Rician K (dB)	0.550	-1.394	9.606	-0.636	0.79	4.30

Table 1: Class means and separation $|\mu_{\text{LOS}} - \mu_{\text{NLOS}}| / \sigma_{\text{pooled}}$.

At 30 dB the features match what the generator puts in, which checks the code: LOS K -factor 9.61 dB against a 9 dB target, LOS rising time exactly 0 ns for every realisation, LOS delay spread 6.5 times smaller than NLOS. By 0 dB the three amplitude-based features have lost almost all their separation while the two delay-based ones have not, because the τ_l columns carry no noise.

Part (b)

Per SNR: 80/20 stratified split (1600 train, 400 test), `StandardScaler` fitted on the training half only, RBF SVM with $C = 1$.

Classifier	0	5	10	15	20	25	30	Mean
SVM-1 Kurtosis	59.50	82.25	90.75	93.25	93.75	93.50	93.75	86.68
SVM-2 Skewness	58.75	84.50	93.50	95.00	94.50	93.75	95.50	87.93
SVM-3 Rising time	79.75	83.00	81.25	83.00	83.00	82.50	82.50	82.14
SVM-4 RMS delay spread	88.50	90.00	93.00	97.25	98.25	98.25	98.75	94.86
SVM-5 Rician K -factor	64.25	88.00	96.75	97.25	98.00	98.25	98.25	91.54
SVM-6 All five	88.75	98.00	99.50	99.75	100.00	100.00	100.00	98.00

Table 2: Test accuracy (%) vs SNR (dB), 400 test samples per cell.

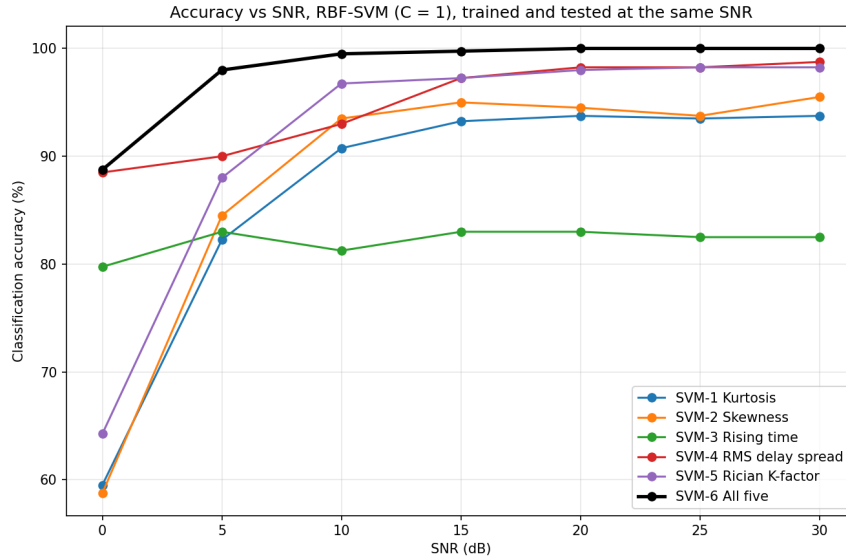


Figure 1: Q1(b)(2).

(i) Best single feature, and why.

RMS delay spread: best mean accuracy (94.86%) and well ahead at low SNR, 88.50% at 0 dB against 79.75% for the next best. Two reasons. The generator draws LOS delays from $\text{Exp}(30 \text{ ns})$ with a 30 ns decay constant and NLOS delays from $\text{Exp}(100 \text{ ns})$ with a 100 ns constant, so LOS power sits within a few ns of the first tap while NLOS power is smeared over hundreds of ns (6.98 ns vs 45.19 ns at 30 dB). And the τ_l values carry no noise, so AWGN reaches τ_{rms} only through the power weights, while kurtosis, skewness and the K -factor are built from the noisy amplitudes alone. Part of this margin is an artefact of the generator using a 3.3 times gap in mean delay; on the paper's measured data the delay spread is useful but not dominant.

(ii) Does combining help, and where most.

Yes, at all seven SNRs. Against the best single feature the gain is +0.25, +8.00, +2.75, +2.50, +1.75, +1.75, +1.25 pp from 0 to 30 dB, so it is most noticeable at 5 and 10 dB. There the features are degraded but fail in different ways, the delay features still informative while the amplitude ones are noisy, so their errors are partly independent. At 0 dB the amplitude features are near chance and add nothing (+0.25 pp); above 20 dB a single feature already reaches 98% so there is little room left.

(iii) Physically meaningful but poor at low SNR.

The Rician K -factor. It is the most direct LOS indicator here and is joint best at high SNR (98.25% at 25 dB), but falls to 64.25% at 0 dB, the largest drop of the five. $\hat{K} = P_{\max}/(P_{\text{total}} - P_{\max})$ has a numerator of one strong tap that noise barely moves, and a denominator of five weak taps carrying only $1/(K+1) \approx 0.11$ between them. At 0 dB the noise power equals the channel power, so the denominator measures noise rather than diffuse power and the estimate collapses to a common value for both classes: 0.55 dB vs -1.39 dB in Table 1, a 2 dB gap against 10 dB at 30 dB. Kurtosis and skewness break the same way and are slightly worse in absolute terms at 0 dB.

Part (c)

SVM-6 trained once on the 25 dB training split, then scored on the test split of every SNR. The scaler is part of the fixed model, fitted at 25 dB and carried over, since a receiver cannot re-fit normalisation on data it has not seen.

Test SNR (dB)	0	5	10	15	20	25	30
Train = Test SNR	88.75	98.00	99.50	99.75	100.00	100.00	100.00
Train at 25 dB	59.50	80.75	97.25	99.50	99.75	100.00	100.00

Table 3: SVM-6 accuracy (%), matched vs mismatched training SNR.

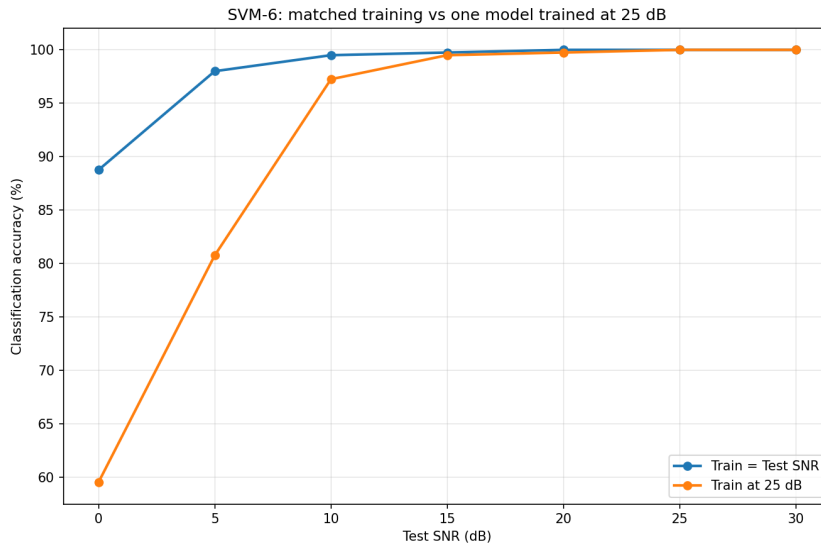


Figure 2: Q1(c)(2).

(i) High SNR agreement.

They agree: identical at 25 and 30 dB, and 0.25 pp apart at 15 and 20 dB, which is one test sample out of 400. So the features have converged by about 15 dB, where the noise contribution

is small next to the class difference and the distributions at 15, 20, 25 and 30 dB are effectively the same. A boundary learned at any of them is the right boundary at all of them, so retraining per SNR gains nothing in that range.

(ii) Low SNR, and why the high-SNR model fails there.

Matched training wins by 29.25 pp at 0 dB and 17.25 pp at 5 dB. Noise does not only add scatter, it shifts the two classes by different amounts: equal noise power on every tap lifts the weak taps of a LOS realisation, so its kurtosis, skewness and K -factor fall and its delay spread rises (20.47 ns at 0 dB vs 6.98 ns at 30 dB). All of that moves LOS points towards the NLOS region, while NLOS features hardly move. The 25 dB boundary was placed for the clean distributions, so it labels the drifted LOS points as NLOS. The confusion matrix at 0 dB confirms it: NLOS 200/200 correct, LOS only 38/200. The fixed scaler adds to this, since features standardised with 25 dB statistics are off-centre at 0 dB.

(iii) Training at 0 dB instead.

It would be different, and not symmetric. The drift is in the same direction but now helps: a boundary fitted at 0 dB sits in the overlapped region, and as the test SNR rises LOS points move away from the NLOS cloud, i.e. further onto the correct side. So the fall-off should be much gentler than in the 25 to 0 dB direction. But the ceiling should be low, because at 0 dB the classes genuinely overlap and the boundary is fitted where labels and features disagree, and because the 0 dB scaler mis-scales high-SNR features (the LOS K -factor mean moves from 0.55 dB to 9.61 dB). I ran it as a check: the 0 dB model stays between 88.75% and 92.25% across the whole sweep, against 59.50% to 100.00% for the 25 dB model. Flatter, but capped.

2 Q2: constellation demodulation and clustering with K-means

Dataset

Grid $\{-3, -1, 1, 3\}^2$, raw average energy $2 \cdot \frac{9+1+1+9}{4} = 10$, so every point is divided by $\sqrt{10}$ for unit energy, leaving a symbol spacing of $2/\sqrt{10} = 0.6325$. With $E_s = 1$ the noise power is $1/\text{SNR}_{\text{lin}}$ split over I and Q, so $\sigma^2 = 1/(2\text{SNR}_{\text{lin}})$ per component. 200 samples per point per SNR gives $16 \times 200 \times 7 = 22400$ rows, seed 67. The generator's check reports mean symbol energy 1.0000 and measured SNR within 0.1 dB of target throughout.

Part (a)

$r = \sqrt{Rx_I^2 + Rx_Q^2}$ and $\theta = \arctan(Rx_Q/Rx_I)$, both added as columns. θ uses `np.arctan2`, not a plain `arctan` of the ratio: a plain `arctan` folds four quadrants onto two, which would give $(0.316, 0.316)$ and $(-0.316, -0.316)$ the same phase and make them inseparable.

The noiseless grid has only 3 distinct radii (0.4472, 1.0000, 1.3416, with 4, 8 and 4 points) and 12 distinct phases, since $(1, 1)$ and $(3, 3)$ share 45° and likewise in the other quadrants. Neither feature identifies a symbol alone; the pair does, with the 45° -type pairs separated by radius only.

Part (b)

All of Part (b) uses the 25 dB subset, 3200 samples.

K	2	4	8	12	14	15	16	17	18	20
Inertia	1926.79	651.34	328.89	168.10	89.08	49.13	10.22	9.95	9.72	9.27
Silhouette	0.389	0.510	0.542	0.677	0.768	0.822	0.880	0.847	0.812	0.742

Table 4: Selected rows of the $K = 2 \dots 20$ sweep; the full sweep is in the notebook.

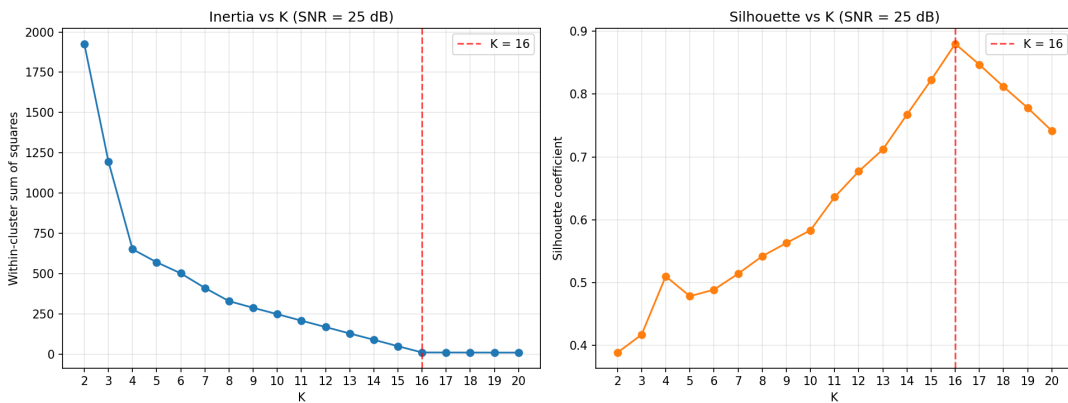


Figure 3: Q2(b)(2).

Both criteria pick $K = 16$ without being told the modulation order. Inertia drops from 49.13 at $K = 15$ to 10.22 at $K = 16$ and then flattens (9.95, 9.72, 9.48, 9.27), because once K matches the number of constellation points each cluster is one noise cloud and the rest of the inertia is just the noise variance inside them; a further centroid can only split a cloud. Silhouette peaks at 0.880 at $K = 16$ and falls after.

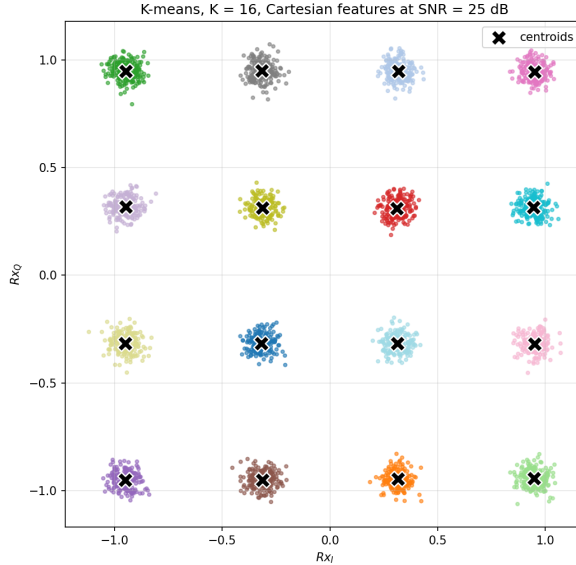


Figure 4: Q2(b)(3). Points coloured by cluster, black crosses are the learned centroids.

Each cluster is one constellation point, and every learned centroid lands within 0.007 of a true symbol against a spacing of 0.6325, so at 25 dB K-means recovers the transmitted grid almost exactly.

(i) Cluster purity for the three feature sets.

At 25 dB all three give 100.00%. That is not a bug: the per-component noise deviation there is $1/\sqrt{2 \cdot 10^{2.5}} = 0.040$ against a symbol spacing of 0.6325, so the clouds are about sixteen standard deviations apart and any sensible representation separates them. To get a ranking I repeated the comparison across the sweep, where Set 1 (Cartesian) leads everywhere the problem is not saturated:

SNR (dB)	0	5	10	15	20	25	30
Set 1 Cartesian	24.25	42.16	77.34	98.34	99.97	100.00	100.00
Set 2 Polar	23.88	41.38	67.38	96.59	99.97	100.00	100.00
Set 3 Both, scaled	23.66	39.50	70.69	96.88	99.97	100.00	100.00

Table 5: Cluster purity (%). Set 3 is standardised before clustering, as required.

The clearest gap is at 10 dB, 77.34% against 67.38% and 70.69%. So the answer is Cartesian, with the caveat that at 25 dB alone the three tie.

(ii) Why Euclidean distance suits Cartesian coordinates but not raw polar ones.

The noise is isotropic in the I/Q plane, so each symbol sits inside a circular Gaussian cloud. Euclidean distance in (Rx_I, Rx_Q) has circular level sets too, so minimising it is nearest-symbol detection, which is the ML rule for this channel, and a QAM constellation is a uniform square lattice, the geometry Lloyd’s algorithm converges to anyway.

In raw (r, θ) three things break. First, r spans about 0.45 to 1.34 while θ spans $-\pi$ to π , five times wider, so unweighted distance counts phase errors far more heavily and stretches clusters into radial slices. Second, angular resolution depends on radius: a displacement d perpendicular to the radius changes the phase by about d/r , so inner points at $r = 0.447$ see three times the phase swing of outer points at $r = 1.342$, and one isotropic metric cannot fit clouds of such different shapes. Third, the phase wraps: $\theta = +\pi$ and $-\pi$ are the same direction but 2π apart

in the feature space, so samples straddling the negative real axis are split between two clusters the algorithm thinks are maximally far apart. Scaling in Set 3 fixes only the first of these, which is why it beats raw polar at 10 dB but still trails Cartesian.

Part (c)

SNR (dB)	0	5	10	15	20	25	30
Adaptive K-Means	24.25	42.16	77.34	98.34	99.97	100.00	100.00
Fixed Template (25 dB)	26.06	47.53	77.53	98.31	99.97	100.00	100.00
ML detector (reference)	26.00	47.69	77.34	98.38	99.97	100.00	100.00

Table 6: Cluster purity (%). The last row is a nearest-constellation-point detector that knows the true grid, included as an upper reference.

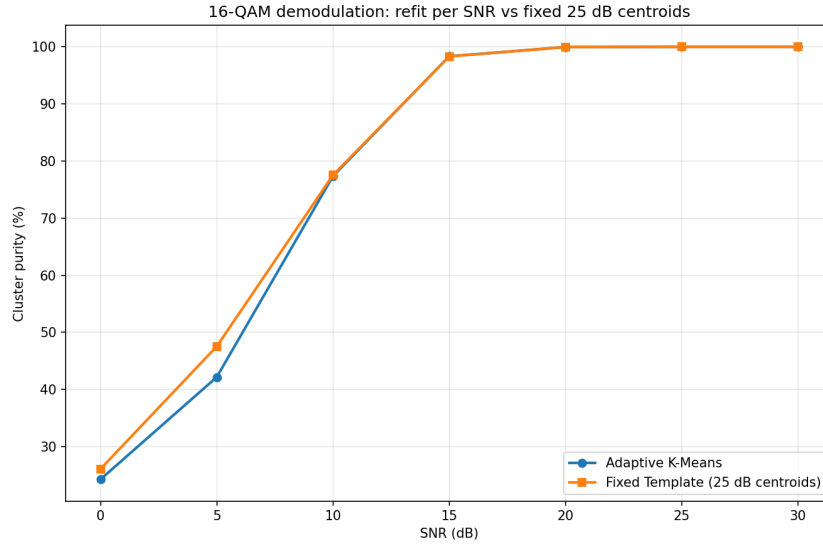


Figure 5: Q2(c)(2).

(i) High SNR agreement and centroid stability.

The curves match: 100.00% at 25 and 30 dB, 99.97% at 20 dB, 0.03 pp apart at 15 dB. Above about 15 dB the clouds are separated by many noise deviations, so a fresh fit lands on the same centroids the template already holds, within 0.007 of the true symbols. The centroids are therefore a property of the modulation, not of the noise level, and re-estimating them at every operating point is wasted work.

(ii) Low SNR.

The fixed template, by 5.37 pp at 5 dB and 1.81 pp at 0 dB. The absolute numbers are low for a reason unrelated to clustering: the ML detector row, which knows the true grid, only reaches 26.00% at 0 dB, because there the per-component noise deviation (0.707) exceeds the symbol spacing (0.6325). The template essentially attains that bound (26.06%), so it is the adaptive fit that is losing something.

(iii) Why fitting on low-SNR samples merges centroids.

K-means minimises the within-cluster sum of squares of the points it is given and knows nothing about the underlying grid. At low SNR the 16 clouds have merged into one roughly

Gaussian blob, so there are no longer 16 separated modes to find, and Lloyd's algorithm instead cuts that single blob into 16 Voronoi cells of roughly equal mass. Two things follow, both visible in the run. At 0 dB the adaptive clusters claim only 11 of the 16 true symbols as their majority label, against 15 for the template, so several centroids land on the same symbol while five symbols own no cluster and every sample of those five is wrong by construction. And the centroids get pushed outwards: mean radius 1.397 for the 0 dB fit against 0.947 for the true constellation, because a blob of noisy points is wider than the grid that produced it and equal-mass partitioning drags centroids into the tails. The innermost four symbols, closest together in absolute terms, merge first. The template avoids all of this because it is never estimated from noisy data: its centroids still sit on the true grid, so using them is just nearest-symbol detection.